

# **Addis Ababa University**

**College of Natural and Computational Sciences**

**School of Information Science**

## **Mobile Computing**

**Group Assignment – Addis Routes (Route Addis)**

### **Group Members**

|                            |                    |
|----------------------------|--------------------|
| <b>1. Abiy Hailu</b>       | <b>UGR/2461/16</b> |
| <b>2. Eden Gosaye</b>      | <b>UGR/8305/16</b> |
| <b>3. Ehsan Mujib</b>      | <b>UGR/9110/16</b> |
| <b>4. Ephrata Belachew</b> | <b>UGR/1633/16</b> |
| <b>5. Ertiban Debebe</b>   | <b>UGR/3920/16</b> |
| <b>6. Marsilas Addisu</b>  | <b>UGR/8249/16</b> |
| <b>7. Meklit Mesfin</b>    | <b>UGR/7509/16</b> |

# Project Title and Description

## Addis Routes: A digital interactive route guide for Addis Ababa

### Description

Our project aims to build a React Native mobile app that turns manually gathered public transportation data in Addis Ababa into a digital and interactive tool. The app will let users choose where they are starting and where they want to go and will then show the possible routes. It will highlight key stops and give estimated fares based on curated, real world data. By combining route guidance and local knowledge, the app will provides an easy and reliable way for people to get around in an environment where digital resources are limited.

## Code of Important Components

### 1. Tab Bar Component (AppTabs)

The Bottom Tab Navigation component

```
import { Tabs } from 'expo-router';
import { Platform } from 'react-native';
import { Icon } from 'react-native-paper';
import { useSafeAreaInsets } from 'react-native-safe-area-context';
import { useAppTheme } from '@/hooks/use-theme';

export default function AppTabs() {
  const theme = useAppTheme();
  const insets = useSafeAreaInsets();

  return (
    <Tabs
      screenOptions={{
        headerShown: false,
        tabBarActiveTintColor: theme.colors.primary,
        tabBarInactiveTintColor: theme.colors.onSurfaceVariant,
        tabBarStyle: {
          backgroundColor: theme.colors.surface,
          borderTopColor: theme.colors.outline,
          borderTopWidth: 1,
          paddingTop: 4,
          height: Platform.select({
            ios: 60 + insets.bottom,
            android: 60 + insets.bottom,
            default: 64,
          }),
          paddingBottom: insets.bottom > 0 ? insets.bottom : 8,
        },
        tabBarLabelStyle: {
          fontSize: 11,
          fontWeight: '600',
          marginTop: 2,
        },
      }}
    />
  );
}
```

```

<Tabs.Screen
  name="index"
  options={{
    title: 'Home',
    tabBarIcon: ({ color, size }) => (
      <Icon source="compass" size={size} color={color as string} />
    ),
  }}
/>

<Tabs.Screen
  name="routes"
  options={{
    title: 'Routes',
    tabBarIcon: ({ color, size }) => (
      <Icon source="source-branch" size={size} color={color as string} />
    ),
  }}
/>

<Tabs.Screen
  name="saved"
  options={{
    title: 'Saved',
    tabBarIcon: ({ color, size }) => (
      <Icon source="bookmark" size={size} color={color as string} />
    ),
  }}
/>

<Tabs.Screen
  name="about"
  options={{
    title: 'About',
    tabBarIcon: ({ color, size }) => (
      <Icon source="information-outline" size={size} color={color as string} />
    ),
  }}
/>
</Tabs>
);
}

```

## 2. Route Display Card (RouteCard)

A component used to display a route in a card.

```

import { RouteResult } from '@data/types';
import { useAppTheme } from '@hooks/use-theme';
import { useState } from 'react';
import { View } from 'react-native';
import { Card, Chip, Divider, Icon, IconButton, Text } from 'react-native-paper';

interface RouteCardProps {
  route: RouteResult;
}

```

```

index: number;
onSave?: (route: RouteResult) => void;
isSaved?: boolean;
}

export function RouteCard({ route, index, onSave, isSaved }: RouteCardProps) {
  const theme = useAppTheme();
  const [expanded, setExpanded] = useState(false);

  const firstStop = route.segments[0].fromStop;
  const lastStop = route.segments[route.segments.length - 1].toStop;

  return (
    <Card
      mode="outlined"
      onPress={() => setExpanded(!expanded)}
      style={{ marginBottom: 12 }}
    >
      <Card.Content style={{ gap: 12, paddingTop: 16 }}>
        <View style={{ flexDirection: 'row', alignItems: 'flex-start', gap: 12 }}>
          <View style={{ width: 32, height: 32, borderRadius: 8, alignItems: 'center', justifyContent:
'center', backgroundColor: theme.colors.primaryContainer }}>
            <Text variant="labelMedium" style={{ color: theme.colors.primary, fontWeight: '700' }}>
              {index + 1}
            </Text>
          </View>
          <View style={{ flex: 1, gap: 8 }}>
            <View style={{ flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between' }}>
              <View style={{ flexDirection: 'row', alignItems: 'center', flexWrap: 'wrap', gap: 6, flex: 1,
marginRight: 8 }}>
                <Text variant="titleSmall" style={{ fontWeight: '700', flexShrink: 1 }}>{firstStop.name}</Text>
                <Icon source="arrow-right" size={14} color={theme.colors.onSurfaceVariant} />
                <Text variant="titleSmall" style={{ fontWeight: '700', flexShrink: 1 }}>{lastStop.name}</Text>
              </View>
              {onSave && (
                <IconButton
                  icon={isSaved ? 'bookmark' : 'bookmark-outline'}
                  size={24}
                  iconColor={isSaved ? theme.colors.accent : theme.colors.onSurfaceVariant}
                  onPress={() => onSave(route)}
                  style={{ margin: -8 }}
                />
              )}
            </View>
          <View style={{ flexDirection: 'row', flexWrap: 'wrap', gap: 6 }}>
            {route.transferCount > 0 ? (
              <Chip
                compact
                mode="flat"
                icon="swap-horizontal"
                style={{ backgroundColor: theme.colors.secondaryContainer }}
              >
                {route.transferCount} transfer{route.transferCount > 1 ? 's' : ''}
              </Chip>
            ) : null}
          </View>
        </Card.Content>
      </Card>
    )
  }
}

```

```

    ) : (
      <Chip
        compact
        mode="flat"
        icon="check-circle"
        style={{ backgroundColor: theme.colors.tertiaryContainer }}
      >
        Direct
      </Chip>
    )}
  </View>
</View>
</View>

<View style={{ flexDirection: 'row', alignItems: 'center', paddingTop: 4 }}>
  {route.segments.map((seg, i) => (
    <View key={i} style={{ flex: 1, flexDirection: 'row', alignItems: 'center' }}>
      {i === 0 && (
        <View style={{ width: 10, height: 10, borderRadius: 5, backgroundColor: theme.colors.primary }}
        />
      )}
      <View style={{
        flex: 1,
        height: 3,
        borderRadius: 2,
        backgroundColor: seg.transportType === 'bus' ? theme.colors.bus : theme.colors.taxi,
      }} />
      {i < route.segments.length - 1 && (
        <View style={{ width: 14, height: 14, borderRadius: 7, borderWidth: 2, borderColor:
theme.colors.surface, backgroundColor: theme.colors.accent }} />
      )}
      {i === route.segments.length - 1 && (
        <View style={{ width: 10, height: 10, borderRadius: 5, backgroundColor: theme.colors.success }}
        />
      )}
    </View>
  )}}
</View>
</Card.Content>

{expanded && (
  <>
    <Divider />
    <Card.Content style={{ paddingTop: 16, paddingBottom: 8 }}>
      {route.segments.map((seg, i) => (
        <View key={i}>
          <View style={{ flexDirection: 'row', alignItems: 'flex-start', gap: 12, minHeight: 32 }}>
            <View style={{ alignItems: 'center', width: 16 }}>
              <View style={{
                width: 12,
                height: 12,
                borderRadius: 6,
                marginTop: 3,
                backgroundColor: i === 0 ? theme.colors.primary : theme.colors.accent,

```

```

    }} />
    <View style={{
      width: 3,
      flex: 1,
      minHeight: 30,
      borderRadius: 2,
      marginVertical: 2,
      backgroundColor: seg.transportType === 'bus' ? theme.colors.bus : theme.colors.taxi,
    }} />
  </View>
  <View style={{ flex: 1, paddingBottom: 16, gap: 4 }}>
    <Text variant="titleSmall">{seg.fromStop.name}</Text>
    <View style={{ flexDirection: 'row', alignItems: 'center', gap: 8 }}>
      <Chip
        compact
        mode="outlined"
      >
        {seg.transportType === 'bus' ? 'Bus' : 'Taxi'}
      </Chip>
    </View>
  </View>
</View>

{i === route.segments.length - 1 && (
  <View style={{ flexDirection: 'row', alignItems: 'flex-start', gap: 12, minHeight: 32 }}>
    <View style={{ alignItems: 'center', width: 16 }}>
      <View style={{ width: 12, height: 12, borderRadius: 6, backgroundColor:
theme.colors.success }} />
    </View>
    <Text variant="titleSmall">{seg.toStop.name}</Text>
  </View>
)}
</View>
)}}
</Card.Content>
</>
)}

<View style={{ alignItems: 'center', paddingBottom: 8, paddingTop: 4 }}>
  <Icon
    source={expanded ? 'chevron-up' : 'chevron-down'}
    size={16}
    color={theme.colors.onSurfaceVariant}
  />
</View>
</Card>
);
}

```

### 3. Stop Selector Component (StopSelector)

A component to select a stop for route finding with search functionality.

```

import React, { useState, useMemo, useCallback } from "react";
import { View, FlatList, Platform, KeyboardAvoidingView, Modal, TextInput } from "react-native";
import { Searchbar, Text, TouchableRipple, Chip, Icon, IconButton, Divider } from "react-native-paper";
import { getAllStopsSorted } from "@data/routeData";
import { Stop } from "@data/types";
import { useAppTheme } from "@hooks/use-theme";
import type { AppTheme } from "@constants/theme";

interface StopSelectorProps {
  label: string;
  placeholder: string;
  selectedStop: Stop | null;
  onSelect: (stop: Stop) => void;
  icon: string;
}

const StopItem = React.memo(({ item, theme, onSelect, getStopIcon }: { item: Stop; theme: AppTheme; onSelect: (stop: Stop) => void; getStopIcon: (type: string) => string }) => {
  return (
    <TouchableRipple onPress={() => onSelect(item)} style={{ paddingHorizontal: 24, paddingVertical: 12 }}>
      <View
        style={{
          flexDirection: "row",
          alignItems: "center",
          justifyContent: "space-between",
        }}
      >
        <View
          style={{
            flexDirection: "row",
            alignItems: "center",
            flex: 1,
            gap: 12,
          }}
        >
          <View
            style={{
              width: 32,
              height: 32,
              borderRadius: 8,
              alignItems: "center",
              justifyContent: "center",
              backgroundColor: theme.colors.primaryContainer,
            }}
          >
            <Icon source={getStopIcon(item.type)} size={16} color={theme.colors.primary} />
          </View>
          <View style={{ flex: 1, gap: 2 }}>
            <Text variant="bodyMedium" style={{ color: theme.colors.onSurface }}>
              {item.name}
            </Text>
            {item.note && (
              <Text variant="bodySmall" style={{ color: theme.colors.onSurfaceVariant }} numberOfLines={1}>
                {item.note}
              </Text>
            )}
          </View>
        </View>
      </TouchableRipple>
    )
  );
});

```

```

        </Text>
      )}
    </View>
  </View>
</View>
</TouchableRipple>
);
});

export function StopSelector({ label, placeholder, selectedStop, onSelect, icon }: StopSelectorProps) {
  const theme = useAppTheme();
  const [visible, setVisible] = useState(false);
  const [search, setSearch] = useState("");

  const allStops = useMemo(() => getAllStopsSorted(), []);

  const filtered = useMemo(() => {
    if (!search.trim()) return allStops;
    const q = search.toLowerCase();
    return allStops.filter((s) => s.name.toLowerCase().includes(q) || s.id.toLowerCase().includes(q));
  }, [search, allStops]);

  const handleSelect = useCallback(
    (stop: Stop) => {
      onSelect(stop);
      setVisible(false);
      setSearch("");
    },
    [onSelect],
  );

  const getStopIcon = useCallback((type: string) => {
    if (type === "bus") return "bus";
    if (type === "taxi") return "taxi";
    return "swap-horizontal";
  }, []);

  const renderStop = useCallback(({ item }: { item: Stop }) => <StopItem item={item} theme={theme}
onSelect={handleSelect} getStopIcon={getStopIcon} />, [theme, handleSelect, getStopIcon]);

  return (
    <>
      <TouchableRipple
        onPress={() => setVisible(true)}
        style={{
          borderRadius: 12,
          borderWidth: 1,
          borderColor: theme.colors.outline,
          backgroundColor: theme.colors.surface,
          overflow: "hidden",
        }}
        borderless
      >
        <View

```



```

        style={{
            flexDirection: "row",
            alignItems: "center",
            paddingHorizontal: 16,
            paddingVertical: 14,
            gap: 12,
        }}
    >
    <View
        style={{
            width: 36,
            height: 36,
            borderRadius: 8,
            alignItems: "center",
            justifyContent: "center",
            backgroundColor: theme.colors.primaryContainer,
        }}
    >
        <Icon source={icon} size={18} color={theme.colors.primary} />
    </View>
    <View style={{ flex: 1, gap: 2 }}>
        <Text
            variant="labelSmall"
            style={{
                color: theme.colors.onSurfaceVariant,
                textTransform: "uppercase",
                letterSpacing: 0.5,
            }}
        >
            {label}
        </Text>
        <Text
            variant="bodyLarge"
            style={{
                color: selectedStop ? theme.colors.onSurface : theme.colors.onSurfaceVariant,
            }}
            numberOfLines={1}
        >
            {selectedStop?.name || placeholder}
        </Text>
    </View>
    <Icon source="chevron-down" size={18} color={theme.colors.onSurfaceVariant} />
</View>
</TouchableRipple>

<Modal
    visible={visible}
    animationType="slide"
    transparent
    onRequestClose={() => {
        setVisible(false);
        setSearch("");
    }}
>

```

```

<View
  style={{
    flex: 1,
    justifyContent: "flex-end",
    backgroundColor: "rgba(0,0,0,0.5)",
  }}
>
<TouchableRipple
  style={{
    position: "absolute",
    top: 0,
    bottom: 0,
    left: 0,
    right: 0,
  }}
  onPress={() => {
    setVisible(false);
    setSearch("");
  }}
  rippleColor="transparent"
>
  <View style={{ flex: 1 }} />
</TouchableRipple>
<KeyboardAvoidingView
  behavior={Platform.OS === "ios" ? "padding" : "height"}
  style={{
    borderTopLeftRadius: 20,
    borderTopRightRadius: 20,
    maxHeight: "85%",
    minHeight: "60%",
    backgroundColor: theme.colors.surface,
    width: "100%",
    overflow: "hidden",
  }}
>
  {Platform.OS === "android" && (
    <TextInput
      style={{
        position: "absolute",
        width: 0,
        height: 0,
        opacity: 0,
      }}
      showSoftInputOnFocus={false}
      accessible={false}
      importantForAccessibility="no"
    />
  )}
<View
  style={{
    flexDirection: "row",
    alignItems: "center",
    justifyContent: "space-between",
    paddingHorizontal: 24,
  }}

```

```

        paddingVertical: 8,
      }}
    >
    <Text variant="titleMedium">{label}</Text>
    <IconButton
      icon="close-circle"
      size={24}
      onPress={() => {
        setVisible(false);
        setSearch("");
      }}
    />
  </View>
  <Divider />
  <Searchbar placeholder="Search stops..." value={search} onChangeText={setSearch} style={{
marginHorizontal: 16, marginVertical: 8, elevation: 0 }} />
  <FlatList
    data={filtered}
    keyExtractor={({item}) => item.id}
    renderItem={renderStop}
    style={{ flex: 1 }}
    keyboardShouldPersistTaps="handled"
    ItemSeparatorComponent={Divider}
    initialNumToRender={10}
    maxToRenderPerBatch={10}
    windowSize={3}
    removeClippedSubviews={true}
    ListEmptyComponent={
      <View
        style={{
          alignItems: "center",
          justifyContent: "center",
          paddingVertical: 48,
        }}
      >
        <Icon source="alert-circle-outline" size={48} color={theme.colors.onSurfaceVariant} />
        <Text
          variant="bodyMedium"
          style={{
            color: theme.colors.onSurfaceVariant,
            marginTop: 12,
          }}
        >
          No stops found for "{search}"
        </Text>
      </View>
    }
  />
</KeyboardAvoidingView>
</View>
</Modal>
</>
);
}

```